

PuenteLab - MEMORIA_DESCRIPTIVA

Memoria Descriptiva — PuenteLab

1. Identificación del proyecto

Campo	Valor
Nombre	PuenteLab
Concepto	Bridge Builder Junior
Paquete	com.educalab.puentelab
Versión	1.0.0
Área	Ingeniería civil (educativa)
Público objetivo	10 a 15 años
Plataforma	Android nativo (Kotlin, Jetpack Compose)

2. Problema y justificación

Los materiales educativos sobre ingeniería estructural para preadolescentes suelen ser o bien demasiado abstractos (diagramas y fórmulas sin interacción) o bien juegos genéricos de física sin ningún andamiaje pedagógico. PuenteLab propone un punto intermedio: un **constructor táctil real** (nodos, barras, materiales, roles estructurales) evaluado por un **motor de reglas determinista** que da retroalimentación explicable, envuelto en una progresión de juego (escenarios, insignias, coleccionables) que sostiene el interés a lo largo de varias sesiones sin recurrir a mecánicas de presión (vidas, rankings online, compras).

3. Objetivos

General: ofrecer una experiencia offline, atractiva y pedagógicamente honesta para introducir conceptos básicos de ingeniería de puentes (conectividad, presupuesto, carga, tipos estructurales) a chicos y chicas de 10-15 años.

Específicos: - Modelar un motor de simulación propio, determinista y testeable, que dé retroalimentación educativa (no solo “correcto/incorrecto”). - Ofrecer 45 desafíos con dificultad progresiva repartidos en 5 escenarios temáticos con identidad visual propia. - Persistir todo el progreso, diseños e insignias localmente, sin conexión ni cuentas. - Cumplir los requisitos de privacidad infantil (sin datos personales, sin permisos innecesarios).

4. Público objetivo y consideraciones de diseño

El rango 10-15 admite mecánicas más exigentes que el diseño “preescolar”: lectura de textos breves, comparación de resultados, gestión de presupuesto, sistemas de progreso con múltiples ejes (nivel, insignias, sellos). Se evitó deliberadamente la estética infantil excesiva (colores pastel, personajes tiernos, mecánicas de arrastrar-y-

listo sin consecuencia) en favor de una identidad “estudio de ingeniería” — azul plano, naranja de obra, iconografía técnica — sin perder calidez ni humor (el personaje guía PIVOT).

5. Alcance

Incluye: constructor de puentes con motor de simulación real; 45 desafíos en 5 escenarios; 8 materiales; 6 vehículos; sistema de progreso (XP, niveles, 9 insignias, 12 sellos coleccionables); hasta 15 diseños guardables y duplicables; persistencia completa con Room; onboarding de 4 pantallas; perfil local con alias y avatar (sin datos reales).

Excluye explícitamente: multijugador, rankings online, cuentas o login, compras dentro de la app, anuncios, analítica/telemetría, cálculo de ingeniería estructural profesional (el motor es educativo, no normativo), reconocimiento de voz o cámara (no se usan).

6. Requisitos funcionales (resumen)

- RF-01: El sistema debe permitir crear un perfil local (alias + avatar) sin datos personales.
- RF-02: El sistema debe mostrar 45 desafíos organizados en 5 escenarios con estados visuales (bloqueado/disponible/iniciado/completado/dominado).
- RF-03: El sistema debe permitir construir un puente colocando nodos y barras, eligiendo material y rol estructural (calzada/riostro/cable/torre).
- RF-04: El sistema debe calcular el costo del diseño en tiempo real y compararlo con el presupuesto del desafío.
- RF-05: El sistema debe simular el cruce de un vehículo y determinar si el puente aprueba, con retroalimentación explicable por barra.
- RF-06: El sistema debe persistir el historial de intentos y derivar de él el progreso, XP, nivel e insignias — nunca de forma aleatoria.
- RF-07: El sistema debe permitir guardar hasta 15 diseños, duplicarlos y eliminarlos.
- RF-08: El sistema debe funcionar íntegramente sin conexión a internet.

7. Requisitos no funcionales

- RNF-01: Sin permiso INTERNET declarado.
- RNF-02: Persistencia local con Room/SQLite; ninguna pérdida de datos al cerrar la app.
- RNF-03: Arquitectura MVVM con capas data/domain/ui separadas; la lógica de negocio debe ser testeable sin UI.
- RNF-04: Accesibilidad — estados nunca solo por color, contentDescription en iconografía funcional, objetivos táctiles $\geq 40dp$.
- RNF-05: Sesiones de 5-20 minutos; guardado y continuación automáticos (borrador por desafío).

8. Casos de uso principales

- **CU-01 Completar onboarding:** el usuario nuevo ve 4 pantallas y crea su perfil.
- **CU-02 Explorar escenarios:** el usuario navega los 5 escenarios y ve el estado de cada uno de los 9 niveles.
- **CU-03 Construir un puente:** el usuario coloca nodos/barras, elige materiales, ve el costo en vivo.
- **CU-04 Probar puente:** el usuario dispara la simulación, ve la animación del vehículo y el resultado con retroalimentación.

- **CU-05 Guardar y duplicar diseño:** el usuario guarda un diseño aprobado y luego lo duplica para explorar variantes.
- **CU-06 Revisar progreso:** el usuario consulta su nivel, insignias y sellos coleccionables.

9. Módulos / pantallas

Ver README.md sección “Módulos de la app” — 10 módulos principales sin contar diálogos (resultado de simulación, guardar diseño) ni ajustes triviales (sonido/háptica).

10. Arquitectura

MVVM + Repository sobre data/ · domain/ · ui/. Ver docs/MANUAL_TECNICO.md para el detalle de paquetes, ViewModels y flujo de datos.

11. Datos y modelo

Room con 14 entidades (perfil, materiales, desafíos, diseños, nodos, barras, vehículos, intentos de simulación, resultado por barra, sellos, progreso, insignias). Ver docs/BASE_DE_DATOS.md.

12. Reglas de negocio relevantes

- Un puente aprueba solo si: (a) las dos orillas están conectadas, (b) existe una calzada continua de barras tipo “Calzada” cuya pendiente no excede el máximo del nivel, (c) el costo total no excede el presupuesto, y (d) ninguna barra supera su capacidad de carga.
- La capacidad de una barra depende de: resistencia del material, longitud (penalización de tramo largo sin apoyo), y bonificaciones geométricas reales de triangulación (cercha), forma de arco, y alivio de cable (suspensión) — nunca de aleatoriedad.
- El XP y el nivel se recalculan siempre a partir del **mejor intento aprobado por desafío** en el historial completo (no se puede “farmear” repitiendo un desafío ya superado).
- Las insignias y sellos se derivan del mismo historial real, nunca se marcan manualmente ni al azar.
- Máximo 15 diseños guardados por usuario; duplicar un diseño cuenta contra ese cupo.

13. UX y dirección visual

Paleta “estudio de ingeniería” (azul plano #164A7A, naranja de obra #F2994A, ámbar #F5C34C) más un color de acento por escenario (río, cañón, bosque, ciudad, montaña). Todas las ilustraciones (escenas de escenario, mascota PIVOT, medallones de insignia/sello) se generan con Jetpack Compose Canvas — sin imágenes externas ni URLs. Ver docs/MANUAL_TECNICO.md para la lista de componentes visuales.

14. Privacidad

Ver README.md sección “Privacidad”. Ningún dato personal se solicita ni se transmite (no hay transmisión posible: no hay permiso de red).

15. Pruebas

59 pruebas unitarias en app/src/test: 49 sobre la lógica de dominio (BridgeEngine, ProgressEngine, BadgeEngine — 21+17+11), cuya lógica exacta se verificó además de forma independiente con kotlin real fuera de Gradle (56 aserciones ejecutadas, ver docs/BUILD_REPORT.md), y 10 sobre persistencia Room con Robolectric (escritas contra la API real de Room 2.6.1, no ejecutadas en este entorno — ver docs/BUILD_REPORT.md). Ver docs/MANUAL_TECNICO.md sección de pruebas para el detalle por archivo.

16. Limitaciones conocidas

- La compilación con Gradle/Android no se pudo verificar en el entorno de desarrollo de este encargo (sin SDK de Android ni acceso a Google Maven/distribución de Gradle). Se verificó en su lugar la lógica de dominio de forma independiente. Ver docs/BUILD_REPORT.md.
- El balance numérico exacto de los 45 desafíos se diseñó con una fórmula determinista anclada a un caso verificado manualmente (el nivel más difícil de un escenario); no se simuló exhaustivamente cada uno de los 45 niveles contra el motor. Es un candidato razonable para playtesting y ajuste fino.
- El peso del vehículo se traduce internamente a un nivel de demanda equivalente (LOW/MEDIUM/HIGH) en vez de un valor continuo, para mantener el motor simple y testeable; se documenta como simplificación deliberada.
- “SavedDesign” del enunciado se implementó como un campo (isSaved) de la entidad BridgeDesign en vez de una tabla separada, para no duplicar el modelo de nodos/barras. Documentado en docs/BASE_DE_DATOS.md.

17. Mejoras futuras

- Playtesting real para ajustar la curva de presupuesto/dificultad de los 45 niveles.
- Vista de “optimización” que sugiera automáticamente sustituciones de material más baratas que sigan aprobando (el motor ya expone todos los datos necesarios: capacidad, demanda y costo por barra).
- Exportar/importar diseños como archivo local (sin red) para compartir entre dispositivos del mismo usuario.
- Modo daltonico adicional (paleta alternativa) más allá de los iconos/textos ya presentes en cada estado.

18. Conclusiones

PuenteLab entrega una arquitectura Android completa (MVVM + Room + Compose) con un motor de dominio propio, determinista y verificado, contenido semilla suficiente para sentirse como un producto terminado (45 niveles, 8 materiales, 9 insignias, 12 sellos), e identidad visual íntegramente local. La principal limitación honesta es la imposibilidad de verificar la compilación de Android/Gradle en el entorno de desarrollo disponible; se dejó preparado un workflow de CI para que la compilación real ocurra automáticamente en cuanto el proyecto se suba a GitHub.